Proactive, resource-aware AI for industrial systems

From reacting to failure to pre-empting it

M Usman

An applied view of failure prediction in production.

Where this is heading

Maintenance is moving from reacting to pre-empting.

M Usman · From enterprise data to production AI.

The cost sits in unplanned failure

- More than **55%** of industrial maintenance is still **reactive** (US DOE, 2010), a long-standing baseline that has barely shifted.
- Unplanned downtime is estimated to cost the world's largest manufacturers around **$1.5 trillion a year** (Siemens, 2022).
- The upside is large and measured: IoT-enabled predictive maintenance can cut downtime by around **50%** and maintenance cost by around **25%** (McKinsey Global Institute, 2015), and the predictive-maintenance market is growing at roughly **25% a year**.

> The model is rarely the hard part. The signal beneath it and the edge above it are.

The slow step

A prediction nobody can act on is not prediction

Failures are rare by design, so there is little to learn from. And a forecast only helps if it arrives early enough to act, honest about its uncertainty, and light enough to run on the asset. Most "predictive" programmes stop at an alarm threshold.

> The slow step is the gap between detecting a problem and pre-empting it: a prediction early enough to act on, calibrated, and cheap enough to run where the asset is.

The pipeline, end to end

1 · Sensing
the signal foundation

leaks as: no run-to-failure data

›

2 · Indicators
features that trend with damage

leaks as: features that do not transfer

›

3 · Prediction
remaining life, with uncertainty

leaks as: no usable lead time

›

4 · Edge
run on the asset, in budget

leaks as: too heavy, deployed blind

Each stage has one job, one common way to lose the value, and one discipline that protects it. The next four slides take them in turn.

1 · Sensing: the signal foundation

Instrument the asset, ingest high-rate telemetry, align it on one time base, and turn noisy streams into a clean, labelled record of how the asset behaved on its way to failure.

Practical steps

- Fix the asset hierarchy and tag dictionary first: every signal carries its asset, unit, and meaning
- Ingest into a time-series store through a broker (OPC-UA, MQTT), decoupled from processing
- Resolve timestamps and synchronisation explicitly; log jitter rather than interpolating it away
- Gate on sensor-data quality: flat-lines, gaps, clipping, and slow calibration drift

The pitfall: no run-to-failure history. Well-maintained assets rarely fail, so the archive is almost all healthy operation with few failure labels. A model cannot be trained, or honestly tested, on data with almost no failures; sensor drift then manufactures fake trends.

What good looks like. Treat the labelled run-to-failure record as this stage's primary output, ahead of any model: seeded-fault rigs, accelerated-life tests, and disciplined work-order reconciliation.

Illustrative standards and tools: ISO 13374 / OSA-CBM · ISO 14224 · OPC-UA, MQTT · InfluxDB, TimescaleDB · the NASA C-MAPSS benchmark.

2 · Indicators: features that trend with damage

Convert clean signals into physically meaningful condition indicators and health indices that trend with damage, the inputs a prognostic model can reason about.

Practical steps

- Match the domain to the physics: time-domain statistics, FFT for fault orders, wavelets for transients
- Use envelope (demodulation) analysis to expose bearing and gear defect frequencies buried in noise
- Fuse indicators (PCA, Mahalanobis distance, autoencoder error) into one degradation curve
- Select features by monotonicity and prognosability, and normalise within operating regime

The pitfall: features that do not transfer. A health index tuned on one machine and one load encodes the regime, not the damage, and collapses when the duty cycle changes. The matching trap is using whole-trajectory statistics, which leaks the future backwards.

What good looks like. Physically interpretable, regime-normalised features, validated by holding out whole machines and whole operating conditions, computed strictly causally with no look-ahead.

Illustrative standards and tools: ISO 13373 (vibration) · envelope and order analysis, wavelets (scipy.signal, PyWavelets) · tsfresh feature banks.

3 · Prediction: remaining life, with uncertainty

Turn health indicators into a decision: an anomaly score, a failure mode, and a remaining-life estimate with calibrated uncertainty, despite very few real failures.

Practical steps

- Start with semi-supervised anomaly detection on healthy data; classify only confirmed modes
- Frame remaining-useful-life deliberately, and cap the target while the asset is healthy
- Handle imbalance honestly: cost-sensitive learning, resample training folds only, report PR-AUC
- Quantify uncertainty near end-of-life; evaluate with prognostic horizon and alpha-lambda, not RMSE alone

The pitfall: an offline winner with no lead time. A model can post a low remaining-life error yet be accurate only in the last few cycles, past the point where anyone can still act.

What good looks like. Evaluate against the decision: the prediction must clear its bound early enough to fit the asset's failure window and the maintenance lead time, with intervals that widen as data thins.

Illustrative tools: the NASA Prognostics Metrics Library (prognostic horizon, alpha-lambda) · the C-MAPSS benchmark · conformal prediction, deep ensembles · cost-sensitive and one-class learners.

4 · Edge: run on the asset, in budget

Run inference on or beside the asset under hard constraints, monitor the model in service, and close the loop to a maintenance action.

Practical steps

- Write the budget down first: deadline, memory and power ceiling, target hardware (MCU/CPU/GPU/FPGA)
- Compress to fit (INT8 quantisation, pruning, distillation), then measure on the real silicon
- Schedule inference so it never steals the control loop's deadline
- Monitor data and sensor drift on the asset; retrain on threshold, feed the outcome back as a label

The pitfall: too heavy for the edge, deployed blind. A model too slow or too large is quietly down-sampled or exiled to the cloud, defeating the latency case; shipped without monitoring, it decays as sensors age.

What good looks like. Co-design the model and the target so measured latency, memory, and energy sit inside the budget; schedule it safely; monitor drift; wire every prediction to a graded action.

Illustrative tools: TF Lite for Microcontrollers, ONNX Runtime, Apache TVM · INT8 quantisation, pruning, distillation · on-asset drift monitors · the OSA-CBM advisory-generation block.

The maturity ladder

5 · PRE-EMPT · proactive, resource-aware, closed-loop

on-asset prediction inside the failure window, wired to the work order, feeding back a new label

4 · PREDICT · remaining-life prognostics

a remaining-life number with lead time; but is it early enough, and does anyone act on it?

3 · MONITOR · condition-based

act when a threshold is crossed: detection, not prediction; no lead-time estimate

2 · SCHEDULE · time-based

service on a fixed interval; good parts replaced early, surprises in between

1 · REACT · run-to-failure

fix it when it breaks; all overtime and expedited parts

Most teams that say "predictive" are on rung 3 or early rung 4. Value is created at rung 5: early enough, honest about uncertainty, cheap to run on the asset, and closed into an action.

Four pitfalls that span the whole pipeline

- **No run-to-failure data.** Well-maintained assets rarely fail, so there is little to train or honestly test on; without seeded-fault rigs or public benchmarks, every later stage is built on sand.
- **Evaluating without lead time.** A model judged on aggregate error can be accurate only once failure is imminent; measured by prognostic horizon against the maintenance lead time, that performance disappears.
- **Ignoring the edge budget.** A model designed without the deadline, memory, and target hardware as constraints is too heavy to deploy, and gets down-sampled or exiled to the cloud.
- **No feedback loop.** A prediction that lands in a dashboard but raises no graded action creates no value, and a model never re-confirmed against outcomes cannot improve.

The lens behind this view

My doctoral research takes this on directly: predicting failure early enough to act, when failures are rare, deadlines are hard, and the compute budget on the asset is small. That work runs across heterogeneous CPU, GPU, and FPGA hardware.

The hard parts in this deck (rare failures, the lead-time test, the edge budget) are problems I have had to solve, not just describe.

> The research and the engineering behind it are public: mtusman-ai.github.io/M.Usman

In one line

Industrial AI does not fail at the model; it fails at the signal beneath it and the edge above it. Build the labelled foundation, predict with enough lead time, run inside the budget on the asset, and close the loop. That is the climb from reacting to pre-empting.

Sources

US DOE, O&M Best Practices Guide (2010): 55%+ of maintenance still reactive; 8–12% saving over preventive · McKinsey Global Institute (2015): ~50% downtime and ~25% maintenance-cost reduction · Siemens, True Cost of Downtime (2022): ~$1.5tn estimated unplanned-downtime cost · predictive-maintenance market ~mid-20s% CAGR, edge AI ~18% CAGR (analyst-house range) · Method grounded in ISO 13374 / OSA-CBM, the NASA C-MAPSS prognostics benchmark, and the Saxena–Goebel prognostic metrics.

M Usman · From enterprise data to production AI. · mtusman-ai.github.io/M.Usman

See the full portfolio

The work behind this deck sits on one page: the Insights series, the projects and the code that runs them, and the publications.

mtusman-ai.github.io/M.Usman

M Usman · From enterprise data to production AI.